

seqTools Shiny

生物信息学分析平台 —— 支持 RNA-seq、单细胞等组学数据的导入、分析和可视化。

架构特点

- 1. 控制层平行: AI 助手和 UI 界面共同操作同一个 engine\$project (R6 对象), 用户可随意切换操作方式
- 2. 分析层模块化: 所有分析工具通过 ToolRegistry 注册, 支持热加载 (reload), AI 可动态编写新工具
- 3. Engines 纯计算层: 算法实现与状态管理解耦, 便于单元测试和复用
- 4. 异步操作: Shiny 非阻塞, Python WebSocket 异步处理 AI 请求

项目结构

```
seqTools_Shiny/  
├─ start.R           # 统一启动器 (推荐)  
├─ start.bat         # Windows 批处理启动  
├─ start_debug.bat   # 调试模式 (Python 前台运行)  
├─ stop.bat          # 停止所有进程  
├─ app.R             # Shiny 入口  
├─ global.R          # 全局配置和加载  
├─ python/           # Python AI 后端  
│   ├── main.py      # FastAPI 服务器  
│   ├── ai_client.py  # 流式 AI 客户端  
│   ├── r_bridge.py   # R 计算引擎桥接  
│   └─ requirements.txt # Python 依赖  
├─ R/  
│   ├── core/        # 核心层 (R6 对象)  
│   │   ├── project.R # Project R6 类  
│   │   ├── engine.R  # AnalysisEngine R6 类  
│   │   ├── registry.R # ToolRegistry R6 类  
│   │   └─ import_registry.R # ImportRegistry R6 类  
│   ├── engines/      # 纯计算引擎层  
│   │   ├── normalize.R # 标准化引擎  
│   │   ├── deg.R       # 差异分析引擎  
│   │   ├── pca.R       # PCA 引擎  
│   │   ├── volcano.R   # 火山图引擎  
│   │   ├── gsea.R      # GSEA 引擎  
│   │   └─ filter.R     # 数据过滤引擎  
│   ├── tools/        # 分析工具层 (注册到 ToolRegistry)  
│   │   ├── normalize_tool.R  
│   │   ├── deg_tool.R  
│   │   ├── filter_tool.R  
│   │   ├── pca_tool.R  
│   │   ├── volcano_tool.R  
│   │   ├── gsea_tool.R  
│   │   ├── gseaPlot_tool.R  
│   │   └─ vis_tool.R  
│   ├── imports/      # 导入方法层 (注册到 ImportRegistry)  
│   │   ├── bulk_rna_import.R  
│   │   └─ single_cell_import.R  
│   └─ utils/         # 公共辅助函数
```

```
|   |─ tool_helpers.R
|   └─ ui_helpers.R
|─ modules/          # Shiny UI 模块
|   |─ project.R      # 项目管理
|   |─ data.R         # 数据导入
|   |─ analysis.R     # 分析执行
|   |─ ai_console.R   # AI 助手
|   └─ settings.R     # 系统设置
└─ utils/            # 旧版兼容层（已废弃）
    |─ config.R
    |─ state.R
    |─ ai.R
    |─ data_read.R
    |─ data_list.R
    |─ processing_tool.R
    └─ tool_list.R
```

启动方式

方式一：R 统一启动器（推荐）

```
# 在 R/RStudio 中
source("start.R")
```

或命令行：

```
Rscript start.R
```

方式二：Windows 批处理

双击 start.bat：

1. 自动启动 Python AI 后端（端口 8765）
2. 自动启动 R Shiny（端口 3838）

方式三：手动分别启动

终端 1（Python 后端）：

```
cd python
python -m uvicorn main:app --host 0.0.0.0 --port 8765
```

终端 2（R Shiny）：

```
shiny::runApp(".", port=3838)
```

调试模式

双击 start_debug.bat：

- Python 前台运行，显示完整日志
- 修改 python/*.py 后自动重载（--reload）

添加新工具

在 R/tools/ 目录下创建 .R 文件，遵循以下模板：

```
TOOL_NAME <- "my_tool"
TOOL_CATEGORY <- "我的分类"
TOOL_OMICS <- "bulk_rna" # 或 NULL 表示通用

TOOL_SCHEMA <- list(
  param1 = list(type = "select", choices = c("a", "b"), default = "a"),
  param2 = list(type = "number", default = 0.05, min = 0, max = 1)
)

TOOL_RUN <- function(inputs, project) {
  # 参数校验
  # 调用 engine_xxx() 或直接使用 R 包
  # 返回 list(data= 结果, messages=c(" 完成信息"))
}

TOOL_OUTPUTS <- list(
  plot = function(result) result, # 如果是 ggplot
  table = function(result) result # 如果是 data.frame
)
```

然后在 UI 中点击  热加载工具，或调用 AI 助手说”重新加载工具”。

添加新导入方法

在 R/imports/ 目录下创建 .R 文件，遵循以下模板：

```
IMPORT_ID <- "my_import"
IMPORT_NAME <- "我的导入"

IMPORT_SCHEMA <- list(
  file = list(type = "file", required = TRUE),
  param1 = list(type = "select", choices = c("a", "b"), default = "a")
)

IMPORT_RUN <- function(file_path, params) {
  # 读取文件
  # 返回 list(data=list(矩阵名 = 矩阵), meta=list(...), omics_type="...")
}
```

配置

首次启动时会自动创建 seqTools_config.ini：

```
[AI]
provider=deepseek
model=deepseek-chat
key=your-api-key
base_url=
```

```
[analysis]
norm_method=log2
pval_cut=0.05
logfc_cut=1
seed=42
dup=mean

[system]
work_dir=/path/to/work_dir
```

依赖

R 包

- shiny, shinyjs, bslib, configr, DT
- seqTools, shinyWidgets, shinyalert
- geneSync, ggraph, aisdK
- future, promises, httpR
- stringr, ggplot2, ggrepel
- org.Hs.eg.db, org.Mm.eg.db (用于基因注释)

Python 包

- fastapi, uvicorn, httpx, websockets

安装:

```
cd python
pip install -r requirements.txt
```

注意事项

1. 唯一状态源: engine\$project (R6) 是全局唯一状态源, state (reactiveValues) 仅用于 UI 渲染缓存
2. 工具热加载: 修改 R/tools/*.R 后点击”热加载工具”即可生效, 无需重启 Shiny
3. AI 与 UI 同步: AI 操作后结果会自动同步到 UI, UI 操作后 AI 可通过 get_state_summary 查看最新状态
4. 异步安全: 所有 engine\$run() 调用都是同步的 (R 单线程), 但 Shiny 通过 invalidateLater 定期同步状态

License

MIT